

Sheet Music Reader

Backend & Pi Client — Operations Manual

Covers the FastAPI backend (Synology NAS, Docker) and the Raspberry Pi e-ink display client. The iOS companion app is being scoped separately and is not yet covered here — see the Roadmap section.

Hardware-confirmed: As of this revision, panel rotation, display contrast, pedal-driven page turns, and content-aware page cropping have all been verified on the physical hardware — this is the first edition written after a real end-to-end test, not from code alone.

1. System Overview

The reader is three pieces talking over the local network: a library backend that stores and serves sheet music, a Raspberry Pi that displays pages on an e-ink panel and reads pedal taps, and (planned) an iOS app for library management from a phone.

- **Backend** — FastAPI + SQLite, running in two Docker containers on a Synology NAS (host name `memoryalpha`). One container serves the API and renders pages; a second, separate container watches a Dropbox-style drop folder and files/renames incoming PDFs automatically.
- **Pi client** — a Raspberry Pi 4B (CM4 planned for the final build) driving a Waveshare 9.7" IT8951 e-paper HAT, mounted in a custom 3D-printed enclosure in portrait orientation, with a PageFlip Dragonfly Bluetooth pedal for hands-free page turns.
- **iOS app** — not yet built; being planned in a separate conversation. This manual will get an iOS section once that work lands.

Data flow for a page turn

Pedal tap → `pedal_input.py` fires a callback → `reader.py` advances the page number and calls the backend → `router/pages.py` renders (or serves a cached render of) that page, cropped to its printed content and sized for the panel → `display_driver.py` thresholds it to black/white and pushes it to the panel with a fast partial refresh.

2. Backend: Starting & Stopping

Everything backend-side runs via Docker Compose from `/volume1/docker/music-server/` on the NAS.

Normal start / stop

```
docker compose up -d # start both containers
docker compose down # stop and remove both containers
docker compose logs -f music-server # tail the API's logs
```

Picking up a change you made

Which command you need depends on *what* changed — this trips people up because both commands look like they should "just restart the thing."

- Edited a **.py file** under `app/` (source is bind-mounted into the container) → `docker compose restart music-server` is enough. The container re-reads the file from disk on restart.
- Changed an **environment variable or the compose file itself** (e.g. `PANEL_WIDTH`) → `docker compose up -d music-server` is required. `restart` reuses the already-running container and will silently keep the old environment.

Watch out: If you change an env var and things don't seem to take effect, this is almost always why — you restarted instead of recreating.

3. Ingesting New Scores

Drop a PDF into the NAS's Dropbox-style folder (`/volume1/docker/music-server/dropbox/`) and the `dropbox-processor` container picks it up automatically — no manual step needed for the common case.

- `dropbox_processor.py` watches the drop folder with `watchdog`.
- `renaming.propose_filename()` and `ingest_core.parse_filename()` figure out composer / title / instrument / opus / volume from the filename and content, and produce a confidence score.
- High confidence → the file is renamed to `Composer-Title-Instrument-Opus-Volume.pdf` and filed under `PDF_DIR/<Category>/<Composer>/`.
- Low confidence → moved to `needs_review/` with a sidecar `.txt` explaining what was ambiguous, for you to file by hand.
- A SHA-256 ledger file prevents the same PDF from being processed twice, even if it's dropped again with a different filename.

Anything placed directly into the PDF library folder (bypassing the dropbox) is picked up the same way by `watcher.py`, which watches `PDF_DIR` itself and also does a full recursive ingest pass on backend startup.

`rename_tool.py` and `setlist_tool.py` at the repo root are standalone scripts for bulk-renaming a batch of files or building a setlist from the command line, outside the API.

4. Database & Page Cache

Database

SQLite in WAL mode at DB_PATH (/db/music.db inside the container). Schema changes live as numbered files under app/migrations/ and are applied automatically on backend startup — never edit the database schema by hand.

- **composer**, **repertoire**, **repertoire_composer** — the catalog.
- **score**, **score_composer** — individual PDFs and their composer(s).
- **setlist**, **setlist_score** — setlists, each entry with a page_start/page_end range into a score.

Page render cache

Rendering a page with PyMuPDF is too slow to redo on every pedal tap mid-performance, so each rendered page is cached to disk the first time it's requested, at:

```
CACHE_DIR/{RENDER_VERSION}/{score_id}/{page_number}.png
```

Hardware-confirmed: The {RENDER_VERSION} segment is new as of this revision. Earlier, the cache key was just score_id/page_number, so when the rendering logic changed (the content-crop fix below), pages that had already been viewed kept serving their old, stale render until the whole cache directory was wiped by hand. RENDER_VERSION is a string in routers/pages.py that gets bumped any time _render_page_png's output changes — bumping it makes every page re-render fresh automatically. Old version directories are harmless leftovers, safe to delete whenever.

How a page is rendered

Each page is rendered at 4× the panel's resolution in grayscale, then PyMuPDF/PIL finds the bounding box of everything that isn't near-white — i.e. the actual printed music, ignoring however much blank margin that particular page happens to have — crops to that box with a small padding margin, and scales the result (via LANCZOS, preserving aspect ratio) to fit the panel's 825×1200 box. A genuinely blank page falls back to the full, blank render rather than crashing on an empty crop.

5. Pi Client: Setup

Hardware: Raspberry Pi 4B (CM4 planned for the final build), Waveshare 9.7" IT8951 e-paper HAT, PageFlip Dragonfly Bluetooth/USB quad pedal.

```
python3 -m venv --system-site-packages .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install git+https://github.com/GregDMeyer/IT8951.git
```

--system-site-packages matters on Raspberry Pi OS 13 (Trixie) — it lets the venv see the apt-installed RPi.GPIO/rpi-lgpio shim and SPI libraries, which aren't pip-installable there. The IT8951 driver itself isn't on PyPI, hence the separate install from GitHub.

Running it

```
python reader.py --score-id 12
```

Subsequent runs resume the last score and page automatically from `READER_STATE_FILE` — omit `--score-id` once state exists.

6. Pi Client: Configuration Reference

Everything lives in `config.py`, overridable via environment variables. Values marked hardware-confirmed have been verified on Leila's actual unit and shouldn't need to change unless the hardware itself changes.

Env var	Default	Notes
MUSIC_SERVER_URL	<code>http://memoryalpha:8000</code>	FastAPI backend on the NAS.
PANEL_VCOM	<code>-1.87</code>	Printed on the panel's ribbon cable — never guess this. Wrong VCOM causes poor contrast/ghosting and can affect panel longevity.
PANEL_SPI_HZ	<code>24000000</code>	SPI clock speed.
PANEL_ROTATE	<code>CW</code>	Compensates for the enclosure mounting the panel in portrait, 90° from its native 1200×825 landscape orientation. Hardware-confirmed correct.
PANEL_BW_THRESHOLD	<code>200</code>	Grayscale pixels this light or lighter become pure white before every draw (rest become pure black) — keeps anti-aliased text/lines from washing out under DU/GC16.
PANEL_FULL_REFRESH_EVENT	<code>10</code>	Page turns between full GC16 refreshes; fast partial DU refreshes happen in between.
PEDAL_DEVICE_NAME_HINT	<code>PageFlip</code>	Substring match against <code>/dev/input</code> device names. Matches a V5+ Dragonfly's Bluetooth name. Pre-V5 units broadcast as "Quad Pedal" instead — check the label on the pedal's underside.
PEDAL_KEY_NEXT / PEDAL_KEY_PREV	<code>KEY_DOWN / KEY_UP</code>	Hardware-confirmed: left pedal pages back, right pedal pages forward.
READER_STATE_FILE	<code>~/.music_reader_state.json</code>	Last (score_id, page_number), so a reboot resumes on the same page.

Find the pedal's actual device name and raw key codes at any time with:

```
python pedal_input.py --probe
```

7. Display & Pedal Behavior

Refresh strategy

Most page turns use a fast DU (1bpp, black/white-only) partial refresh — quick and low-flicker. DU is a lossy waveform, so repeated partial updates accumulate faint ghosting; every `PANEL_FULL_REFRESH_EVERY`-th turn (default 10) does a full GC16 grayscale refresh instead to clear it. The very first page of a session also always forces a full GC16 refresh, since clearing the panel on startup only blanks it to white — it doesn't draw real content.

Anti-aliased text/line edges from the backend's grayscale renders arrive as mid-gray pixel values, but every refresh mode here needs real contrast to read as solid black once printed to actual e-ink pigment — so every page is thresholded to pure black/white (`PANEL_BW_THRESHOLD`) before it's pushed to the panel.

Hardware-confirmed: Rotation, refresh cadence, and black/white thresholding have all been run through a real score on the physical panel, with the pedal driving page turns end to end.

Pedal

The Dragonfly pairs as a generic Bluetooth HID keyboard, so it shows up as a `/dev/input/event*` device like any keyboard. Its primary (larger) pedals never send Left/Right Arrow in any of its five preset modes — only Stop/Play, Up/Down Arrow, or Space/Enter — so this project uses the Up/Down mapping. The pedal auto-sleeps after 30 minutes of inactivity; a tap wakes and reconnects it.

Screen burn-in

Not a concern the way it is for OLED. E-ink pixels are physical pigment particles, not an emissive element that degrades with use, so there's no permanent-damage failure mode. The only related phenomena are temporary: DU ghosting (already handled by the periodic full refresh above) and, for a page left completely static for weeks at a time, mild "image persistence" that clears after a refresh cycle or two. The panel is bistable — it holds its image with zero power — so sitting on the last page between practice sessions is completely normal.

8. Troubleshooting

Symptom	Cause / Fix
Env var change (e.g. PANEL_WIDTH) doesn't seem to take effect	You restarted instead of recreating the container. Run <code>docker compose up -d music-server</code> , not <code>restart</code> .
Backend code change doesn't seem to take effect	Plain <code>docker compose restart music-server</code> is enough for a source-file edit. If the change was to page-rendering logic specifically, also confirm <code>RENDER_VERSION</code> was bumped in <code>routers/pages.py</code> — otherwise already-cached pages keep serving their old render.
<code>RuntimeError: No input device matching 'PageFlip' found</code>	Almost always the pedal's 30-minute auto-sleep — tap it to reconnect. If that doesn't fix it, confirm <code>PEDAL_DEVICE_NAME_HINT</code> matches the pedal's actual Bluetooth name (pre-V5 units broadcast "Quad Pedal" instead of "PageFlip Dragonfly").
Page displays sideways, or only partially fills the panel	<code>PANEL_ROTATE</code> isn't matching the physical mount. Confirmed value for this enclosure is <code>CW</code> ; try <code>CCW</code> if the enclosure is ever rebuilt differently.
Text looks faint or broken up	Check <code>PANEL_BW_THRESHOLD</code> — too far toward 0 drops thin strokes, too far toward 255 can make text too heavy. If it's specifically the very first page of a session, confirm <code>display_driver.py</code> still seeds its full-refresh counter at <code>PANEL_FULL_REFRESH_EVERY</code> , not 0.
One page shows more white space around the music than another	Normal page-to-page variation in how much margin the original engraving/scan leaves — not a bug. The backend already crops each page to its own printed content before fitting it to the panel; a page that genuinely has a shorter final system will still show more blank space below it than a page whose content runs closer to the bottom.
Backend unreachable from the Pi	<code>api_client.py</code> retries connection failures (NAS asleep, Wi-Fi drop) 3× with backoff before giving up, and shows an on-panel error message rather than crashing. Confirm the NAS is awake and reachable at <code>MUSIC_SERVER_URL</code> .

9. Backups & Falling Back

NAS data

- `db/` and `pdf/` under `/volume1/docker/music-server/` are the two directories that actually matter — everything else regenerates. Back these up with a Synology snapshot or a periodic `rsync` copy off the NAS.
- `cache/` and `logs/` are disposable — the cache rebuilds itself on demand and nothing depends on old logs.

Source code

The backend and Pi client source live in a git repo (github.com/HappyMutantLM/music-reader). As of this revision, the first fully hardware-confirmed state is preserved as an annotated tag:

```
git checkout v1-prototype-working
```

A portable, full-history git bundle of that checkpoint (a single file containing the entire repo — clonable on its own, independent of GitHub) is kept in `archives/` inside the repo, and a copy was also handed off outside the repo as an extra off-laptop fallback.

Note: Worth doing periodically: `git push origin main --tags` so GitHub stays a live mirror too, not just this one snapshot.

10. Roadmap & Known Gaps

- **No on-panel score/setlist picker.** A score is chosen via `--score-id` or by hand-editing the state file. A future "browse the library" screen would remove this.
- **13.3" dual-panel upgrade planned.** `display_driver.py` already trusts the IT8951 driver's own reported width/height rather than a hardcoded constant, so this should carry over without a rewrite — worth re-verifying `PANEL_ROTATE` and the content-crop padding once that hardware exists.
- **iOS companion app** — in progress in a separate conversation; will get its own section here once there's something to document.
- **Content-crop tuning.** `CONTENT_WHITE_THRESHOLD` (currently 250) and the crop padding were tuned against one scanned edition (Czerny, Peters). A score scanned with a tinted or noisier background may need the threshold adjusted to crop as tightly.

11. Revision Notes

September 5, 2026 (this revision)

- Rewritten after the first full hardware test — panel rotation, display contrast, and pedal-driven page turns are now confirmed working rather than "should work" or "not yet tested," and the manual's language reflects that throughout.
- Documented the `PANEL_ROTATE`, `PANEL_BW_THRESHOLD`, and `PANEL_FULL_REFRESH EVERY` fixes that resolved the initial 90°-rotation and faint/incomplete-image bugs found on first power-up.
- Documented the pedal migration from AirTurn to PageFlip Dragonfly and the confirmed left=back / right=forward key mapping.
- Added the content-aware page-cropping pipeline in `routers/pages.py` and its self-versioning render cache (`RENDER_VERSION`), which replaces the earlier flat `score_id/page_number` cache key.
- Added the Backups & Falling Back section covering the new git tag/bundle practice.

July 9, 2026 (prior edition)

Original edition, written from code review before any hardware test. Referenced an AirTurn pedal (since replaced), described several already-built features as roadmap items, and stated pages were "fit-and-centered onto a 1200×825 canvas" — since replaced by content-aware cropping onto a portrait 825×1200 box.